

# IT-359 Group Project

Automated Network Reconnaissance  
& Vulnerability Assessment Tool

---

Authors: Dylan Voss & Grant Gollinger

Course: IT-359

Illinois State University

Spring 2026

|                                                                    |
|--------------------------------------------------------------------|
| <b>Tool:</b> recon.sh — Bash-based network scanner                 |
| <b>Technologies:</b> Bash, Nmap, grep, awk, sed                    |
| <b>Purpose:</b> Automate penetration testing reconnaissance phases |

# 1. Introduction & Purpose

---

Network security has become one of the most critical concerns for organizations of all sizes. Penetration testing — the practice of authorized, simulated attacks against a network to identify vulnerabilities before malicious actors can exploit them — is a cornerstone of modern security practice. However, the early phases of a penetration test, known collectively as reconnaissance, are often time-consuming and repetitive. Security professionals must manually discover live hosts, scan for open ports, enumerate services, and assess risk across potentially hundreds of machines. This manual effort introduces the risk of inconsistency and human error.

The IT-359 Group Project, developed by Dylan Voss and Grant Gollinger, directly addresses this problem by delivering **recon.sh**: an automated network reconnaissance and vulnerability assessment tool built using Bash scripting and the Nmap security scanner. The tool automates the full reconnaissance workflow — from discovering which hosts are alive on a network, to identifying open ports, enumerating running services, flagging known security risks, and producing a structured report summarizing all findings with actionable mitigation recommendations.

## 1.1 Project Objectives

The tool was designed with five primary objectives:

- Automate network reconnaissance tasks using custom-written Bash scripts, reducing the manual effort required during the early phases of a penetration test.
- Demonstrate practical penetration testing concepts — specifically host discovery, port scanning, and service enumeration — in a controlled, authorized lab environment.
- Analyze scan results programmatically to identify potential security risks based on known dangerous ports and service configurations.
- Generate clear, structured output reports that include both findings and recommended mitigations, suitable for review by a security analyst.
- Follow ethical and legal standards for security testing, ensuring all activity was conducted only on systems owned by the project team or explicitly authorized for testing.

## 1.2 Scope & Ethical Boundaries

This project was conducted strictly for educational purposes within the context of the IT-359 course. All scanning and enumeration was performed exclusively on lab machines and virtual environments owned by the project members. No public networks, production systems, or unauthorized hosts were targeted at any point. The tool does not include exploitation capabilities — it is strictly limited to reconnaissance and enumeration, which represent the information-gathering phases of the penetration testing lifecycle (i.e., prior to any active attack).

The tool includes a deliberate design choice in support of this ethical posture: it accepts only an explicit target IP address or subnet as a required argument (-t), and will not run without operator-specified authorization. There is no capability for autonomous target selection or unsolicited scanning.

## 2. Technical Implementation

The tool is implemented as a single Bash script (**recon.sh**) that orchestrates Nmap and standard Unix text-processing utilities across five sequential phases. Each phase is encapsulated in its own Bash function, following the principle of modular design. The script accepts a target IP address or CIDR subnet via the **-t** flag and an optional output directory via **-o**.

### 2.1 Tools & Technologies

| Tool / Technology      | Role in the Project                                                                  |
|------------------------|--------------------------------------------------------------------------------------|
| Bash (Shell Scripting) | Primary implementation language; orchestrates all phases and logic                   |
| Nmap                   | Network scanner for host discovery, port scanning, and service enumeration           |
| grep                   | Filters and extracts relevant lines from Nmap output files                           |
| awk                    | Parses whitespace-delimited fields (e.g., extracting IP addresses from scan results) |
| sed                    | String manipulation and output formatting                                            |
| Linux (Ubuntu/Kali)    | Operating system environment; provides native tooling and file I/O                   |

### 2.2 Script Architecture & Execution Flow

The script begins with a dependency check that verifies Nmap, grep, awk, and sed are installed before any scanning begins. A timestamped output directory is created automatically for each run (e.g., *reports/20260421\_143022\_192.168.1.0\_24/*), keeping results from different scans cleanly separated. The script then detects whether the target is a single IP or a subnet by checking for the presence of a CIDR prefix (the */* character), and adjusts its scanning behavior accordingly.

#### Step 1 — Host Discovery

For subnet targets, the script performs a ping sweep using **nmap -sn** (also known as a "no-port-scan" or "ping scan"), which sends ICMP echo requests and TCP/ARP probes to identify which hosts are online. Results are parsed with grep and awk to extract live IP addresses into *live\_hosts.txt*. If no live hosts are found, the script exits gracefully.

For single-host targets (no CIDR prefix), the script skips the ping sweep and uses Nmap's **-Pn** flag (treat host as up, skip host discovery). This is necessary for targets that block ICMP — a common configuration in Hack The Box machines and firewalled enterprise hosts — which would cause a naive ping sweep to incorrectly classify the host as offline.

#### Step 2 — Port Scanning

Open TCP ports are identified with an **nmap -sS** (SYN stealth scan) across all 65,535 ports (**-p-**), filtered to only report open ports (**--open**). Results are saved in both XML format (*raw\_scan.xml*) for structured parsing and grepable format (*ports.gnmap*) for quick text-based filtering. The SYN scan is chosen because it is faster than a full TCP connect scan and less likely to be logged by target services, which is consistent with real-world stealth reconnaissance.

## Step 3 — Service Enumeration

For each host with open ports, the script runs **nmap -sV -sC** to detect service versions (**-sV**) and execute Nmap's default scripting engine scripts (**-sC**). Version detection queries each open port and attempts to identify the software name and version number (e.g., *Apache httpd 2.4.51*, *OpenSSH 7.9*). The scripting engine (**NSE**) runs additional checks such as grabbing SSL certificate information, checking for anonymous FTP access, and retrieving HTTP server headers. Results are saved in both XML and human-readable text formats.

## Step 4 — Risk Identification

The script maintains an associative array (**declare -A RISKY\_PORTS**) mapping 32 well-known port numbers to plain-English risk descriptions. After service enumeration, the script iterates through this dictionary and cross-references each risky port against the service scan output. Any match is written to a risk log (*risks.txt*) with the associated host(s) listed. The risk database covers a broad spectrum including:

- Unencrypted legacy protocols: FTP (21), Telnet (23), rsh (514), rlogin (513)
- Exposed database services: MySQL (3306), MSSQL (1433), MongoDB (27017), Redis (6379), PostgreSQL (5432), Oracle DB (1521), Elasticsearch (9200)
- Windows attack surface: SMB (445), MS-RPC (135), NetBIOS (137/139), RDP (3389)
- Common misconfiguration risks: SNMP (161), NFS (2049), VNC (5900), HTTP (80/8080)
- Backdoor indicators: Port 4444 (Metasploit default listener / potential C2 beacon)

## Step 5 — Report Generation

The final phase consolidates all scan data into a human-readable text report (*report.txt*) organized into five sections: Live Hosts, Service Enumeration, Identified Risks, Mitigation Recommendations, and a reference to raw scan files. The mitigation section provides targeted guidance based on the class of services discovered — for example, recommending replacement of FTP with SFTP, enforcing authentication on exposed database services, patching SMB for EternalBlue, and reviewing TLS configurations on HTTPS endpoints.

## 2.6 Sample Output — Hack The Box: Meow

To validate the tool in a realistic, authorized environment, the team tested it against **Meow**, the first machine in Hack The Box's Starting Point Tier 0. Hack The Box (HTB) is a legal online platform providing intentionally vulnerable machines for security training. Meow is an easy-difficulty Linux machine designed around exploiting an open Telnet service — an ideal test case for demonstrating the tool's risk detection capability.

The tool was run against the HTB VPN target IP **10.129.1.17** in single-host mode (no CIDR prefix), which automatically engaged **-Pn** to bypass ICMP blocks — a necessary flag since HTB machines typically do not respond to ping. The full scan completed in under 21 seconds and correctly identified port 23 (Telnet) as open and flagged it as a high-risk service.

```
===== IT-359 NETWORK
RECONNAISSANCE & VULNERABILITY REPORT Generated: Thu Apr 16 02:27:23 PM CDT 2026
Target: 10.129.1.17 Scan Mode: single (single host, ping disabled (-Pn)) Authors:
Dylan Voss & Grant Gollinger
===== SECTION 1: LIVE HOSTS
----- Total live hosts
found: 1 10.129.1.17 SECTION 2: SERVICE ENUMERATION
----- Nmap scan report for
10.129.1.17 Host is up (0.0094s latency). Not shown: 999 closed tcp ports (reset)
PORT STATE SERVICE VERSION 23/tcp open telnet Linux telnetd Service Info: OS:
Linux; CPE: cpe:/o:linux:linux_kernel Nmap done: 1 IP address (1 host up) scanned
in 20.52 seconds SECTION 3: IDENTIFIED RISKS
----- [RISK] Port 23 -
Telnet - Unencrypted remote access, credential sniffing risk Affected: 10.129.1.17
SECTION 4: MITIGATION RECOMMENDATIONS
----- * Disable or
firewall unencrypted services (FTP, Telnet) * Replace unencrypted protocols with
SSH/SFTP * Apply latest patches for SMB, RDP, Windows RPC services * Review TLS/SSL
configs on HTTPS services for weak ciphers
===== END OF REPORT
=====
```

The output above demonstrates the tool's end-to-end workflow on a real target. The risk identification phase correctly matched port 23 against the *RISKY\_PORTS* dictionary entry for Telnet and attributed it to the correct host IP. On Meow, Telnet is indeed the intended attack vector — the service accepts a root login with no password, which is the flag submission mechanism for the HTB challenge. The tool's ability to surface this vulnerability in a single automated pass validates its core design.

## 3. Justification & Analysis

---

### 3.1 Why Bash?

Bash was selected as the implementation language for several deliberate reasons. First, Bash is natively available on virtually every Linux distribution — including Kali Linux, which is the industry-standard operating system for penetration testers — meaning the tool requires no additional runtime installation beyond Nmap itself. Second, Bash excels at orchestrating command-line tools: calling Nmap, piping output through `grep` and `awk`, and writing results to files are all first-class operations in Bash, achievable in far fewer lines than equivalent Python code that wraps subprocess calls.

Third, writing this tool in Bash reinforces a skill set that is directly applicable in professional security contexts. Security engineers and penetration testers routinely write Bash scripts to automate repetitive enumeration tasks during engagements. By building this project in Bash rather than a higher-level language, the team gains practical experience with the same tooling used in real-world assessments.

A Python implementation would have offered advantages in structured data handling and cross-platform compatibility, and a library like *python-nmap* could provide richer programmatic access to scan results. However, for the scope of this project — focused on Linux-based lab environments with standard Unix tooling — Bash represented the most direct, dependency-light, and professionally relevant choice.

### 3.2 Why Nmap?

Nmap (Network Mapper) is the most widely used and thoroughly documented network scanning tool in the security industry. It has been in continuous development since 1997, supports a vast range of scan types, and its output formats (normal text, grepable, XML) are well-established standards that integrate cleanly with other tools in the penetration testing ecosystem. For an educational project aimed at demonstrating real reconnaissance workflows, Nmap was the only appropriate choice.

Alternatives such as `masscan` offer faster scanning speeds for internet-wide surveys, but lack Nmap's service version detection and scripting engine capabilities. The Nmap Scripting Engine (NSE) in particular adds significant value: by running `-sC` default scripts during service enumeration, the tool automatically performs checks that would otherwise require separate specialized tools (e.g., checking for anonymous FTP login, enumerating SMB shares, reading HTTP server headers). This makes Nmap the single most capable tool for a consolidated reconnaissance pipeline.

### 3.3 Design Decisions & Trade-offs

| Design Decision                      | Rationale                                       | Trade-off                                           |
|--------------------------------------|-------------------------------------------------|-----------------------------------------------------|
| SYN scan (-sS) over full TCP connect | Faster; less likely to be logged by services    | Requires root/sudo privileges                       |
| -p- (all 65535 ports)                | Catches services on non-standard ports          | Significantly slower than top-1000 scan             |
| Single-host -Pn mode                 | Supports firewalled/HTB targets that block ICMP | Cannot confirm host is actually alive               |
| Associative array for risk mapping   | Fast O(1) lookup; easy to extend with new ports | Static list; not updated dynamically from CVE feeds |
| Timestamped output directories       | Preserves history of all scan runs              | Accumulates disk usage over many scans              |
| Text-based report output             | Human-readable; no rendering dependencies       | Less polished than HTML/PDF report output           |

### 3.4 Risk Identification Methodology

The risk identification phase is based on a curated list of 32 ports that represent known security concerns in real-world network assessments. The methodology deliberately prioritizes breadth of coverage over deep vulnerability analysis — the goal is to surface the most impactful risks quickly and flag them for follow-up, not to perform automated exploitation or provide CVE-specific vulnerability scoring.

This approach mirrors how experienced penetration testers triage initial scan results: they first identify the "highest value" open ports (unencrypted services, exposed databases, legacy remote access protocols), then prioritize those for deeper manual investigation. By encoding this expert knowledge into the script's risk dictionary, the tool effectively acts as a first-pass triage layer that a junior analyst can run before deeper engagement.

A limitation of this approach is that it produces false negatives for services running on non-standard ports (e.g., an SSH server moved to port 2222 would not be flagged as risky even if it has a vulnerable version). Future improvements could address this by cross-referencing service version banners against a local CVE database rather than relying solely on port numbers.

### 3.5 AI Integration

The project plan outlined an AI integration component, exploring whether language models could assist in different phases of the reconnaissance workflow — for example, using a code-focused model to assist with script generation or a smaller model to parse and summarize Nmap output. The team evaluated available models including GPT-class and Gemma-based options against the specific needs of each pipeline step.

The practical conclusion was that for structured, rule-based tasks like port risk matching and report generation, traditional scripting logic is more reliable, faster, and fully deterministic compared to LLM-based alternatives. AI models are better suited for the *interpretation* layer — summarizing



findings in natural language, suggesting contextual mitigations, or providing on-demand explanations of discovered services. This distinction informs a natural future extension: adding an optional AI-powered summary step that passes the risk log to a language model API for narrative analysis.

## 4. Conclusion

---

The IT-359 Group Project successfully delivers a functional, modular automated reconnaissance tool that covers the complete early-stage penetration testing workflow. The **recon.sh** script demonstrates that a relatively small amount of well-structured Bash code — combined with the power of Nmap — can meaningfully automate tasks that would otherwise require significant manual effort and specialized knowledge to perform consistently.

The five-phase architecture (host discovery → port scanning → service enumeration → risk identification → report generation) closely mirrors the reconnaissance methodology used in professional penetration testing engagements, making this project directly applicable to real-world security work. The tool handles both single-host and subnet targets, adapts its scanning strategy based on target type, and produces self-contained, timestamped output directories for each run.

### 4.1 Key Outcomes

- Successfully automated all five reconnaissance phases in a single executable script requiring no configuration beyond specifying a target.
- Implemented intelligent target-mode detection (single host vs. subnet) with appropriate scan flags for each case, including support for firewalled hosts via `-Pn`.
- Built a 32-entry risk dictionary covering the most commonly encountered dangerous ports in real-world assessments, with plain-English descriptions and affected-host attribution.
- Generated structured, multi-section reports including raw scan data references, risk summaries, and actionable mitigation recommendations.
- Applied ethical security testing principles throughout, with the tool architected to require explicit operator authorization before any scanning begins.

### 4.2 Limitations

As with any tool built within the scope of a course project, several limitations exist. The risk identification logic is port-number based rather than version-based, meaning a patched service on a risky port will still generate a flag. The tool does not perform UDP scanning, which would be necessary for a complete assessment of services like SNMP and DNS. The text-based report format, while functional, is less polished than the HTML or PDF output that commercial tools produce. Finally, the tool has only been validated in lab environments and would require additional testing before use in professional engagements.

### 4.3 Future Improvements

- Version-based risk scoring: Cross-reference detected service versions against a local CVE database (e.g., using searchsploit) to flag specific known vulnerabilities rather than only

port-level risks.

- UDP scanning support: Add an optional UDP scan phase for comprehensive coverage of services like SNMP (161), TFTP (69), and DNS (53).
- AI-powered summary layer: Pass the risk log and service enumeration output to a language model API to generate a natural-language executive summary of findings.
- HTML/PDF report output: Replace the plain-text report with a formatted HTML or PDF output for more professional deliverables.
- Automated scheduling: Add cron-compatible output and delta-comparison logic to support recurring scans that alert on network changes over time.

Overall, this project demonstrates the value of automation in security workflows and provides a solid foundation upon which a more fully-featured reconnaissance toolkit could be built. The core design — modular Bash functions, Nmap as the scanning engine, structured output files, and a curated risk knowledge base — represents sound engineering that scales well with additional features. Most importantly, it reflects a genuine understanding of penetration testing methodology and the role that automation plays in making security assessments more efficient, consistent, and repeatable.